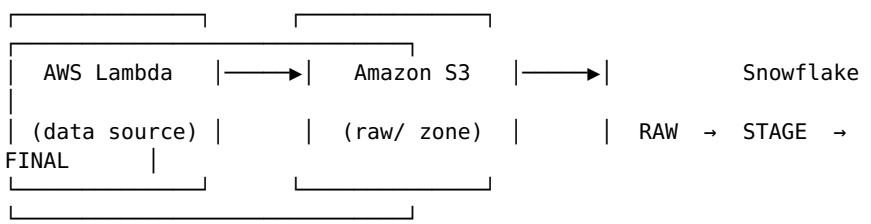
Event-Driven Data Pipeline: Lambda → S3 → Snowflake

An event-driven ELT (Extract, Load, Transform) data pipeline that generates data via **AWS Lambda**, lands it in **Amazon S3**, and loads it into **Snowflake** following a **RAW → STAGE → FINAL** layered architecture.

🏗 Architecture



Stage	Purpose
Lambda	Generates/extracts data and writes JSON files to S3, partitioned by date (`raw/yyyy/mm/dd/`)
S3	Durable landing zone; decouples the producer (Lambda) from the consumer (Snowflake)
Snowflake — RAW	Stores data exactly as received, no transformation. Source of truth for reprocessing.
Snowflake — STAGE	Cleaned, typed, flattened data — individual columns extracted from JSON.
Snowflake — FINAL	Business-ready, aggregated tables used for reporting/analytics.

📁 Project Structure

```
.
├── README.md
├── lambda/
│   └── lambda_function.py       # Generates fake data & uploads to S3
├── sql/
│   ├── 01_storage_integration.sql
│   ├── 02_raw.sql
│   ├── 03_stage.sql
│   └── 04_final.sql
└── docs/
    └── ARCHITECTURE.md          # Detailed setup & design notes
```

⚙ Tech Stack

- **AWS Lambda** (Python 3.11) — data generation/extraction
- **Amazon S3** — data lake landing zone
- **Snowflake** — data warehouse (RAW/STAGE/FINAL layers)
- **IAM Roles + Storage Integration** — secure, keyless S3 ↔ Snowflake connection

---

🚀 Setup Guide

1. Deploy the Lambda function

```
cd lambda
zip function.zip lambda_function.py

aws lambda create-function \
  --function-name fake-data-generator \
  --runtime python3.11 \
  --role arn:aws:iam::<account-id>:role/lambda-s3-writer \
  --handler lambda_function.lambda_handler \
  --zip-file fileb://function.zip \
  --timeout 30 \
  --memory-size 128
```

Update `BUCKET` in `lambda_function.py` with your own S3 bucket name before deploying.

2. Create an S3 bucket

```
aws s3 mb s3://your-bucket-name --region us-east-1
```

3. Run the SQL scripts in Snowflake (in order)

```
01_storage_integration.sql   → connects Snowflake to your S3 bucket
02_raw.sql                   → creates and loads the RAW table
03_stage.sql                 → cleans and flattens data into STAGE
04_final.sql                 → aggregates data into FINAL
```

See docs/ARCHITECTURE.md for full step-by-step instructions, including IAM role/trust policy setup.

📊 Sample Analytics Queries

```
-- Revenue by city
SELECT city, SUM(price * quantity) AS total_revenue
FROM stage_data
GROUP BY city
ORDER BY total_revenue DESC;

-- Best-selling product
SELECT product, SUM(quantity) AS total_sold
FROM stage_data
GROUP BY product
ORDER BY total_sold DESC;
```

🔒 Security Notes

- No AWS keys are hardcoded — Snowflake connects to S3 via a **Storage Integration** (IAM role + external ID trust).
- Lambda uses least-privilege IAM permissions (`s3:PutObject` scoped

to a single prefix).
- Any API keys/secrets should be stored in **AWS Secrets Manager**, never committed to this repo.

🗺 Roadmap / Next Steps

- ☐ Automate RAW → STAGE → FINAL with Snowflake **Streams & Tasks**
- ☐ Add **Snowpipe** for continuous, event-driven ingestion (no manual `COPY INTO`)
- ☐ Add monitoring/alerting (CloudWatch + Snowflake `TASK_HISTORY`)
- ☐ Add a BI layer (dashboard) on top of `final_data`